1st Task

You are responsible for providing semi-platform for developers which help with their application maintaing and speed up deployment process. You are asked to prepare application deployed in Kubernetes environment (manifests only) using GitOps approach. There is a need to write a Helm Chart to help with future application management in additional environments. Files can be shared to us via Github profile (or any other public git repository). You decide how the file structure will look like.

AppName: Spring-Boot API
Image: busybox:stable

Cluster names:

- dev-global-cluster-0
- prd-global-cluster-5

Application Namespace and Argo CD's Project name is your decision.

Time needed for assessment: **24h**

Technologies used: Argo CD, Helm, Kubernetes

Requirements:

Argo CD

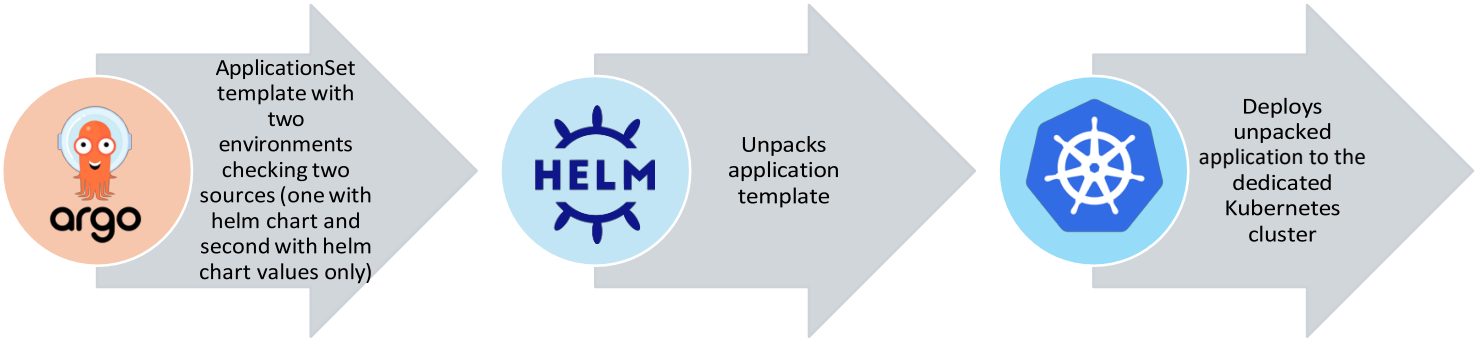
- Use Application Set for this with List Generator
- Application Set should create Namespace if there is no one existing
- Application is being unpacked using Helm Chart (can be either public, or imagined – not existing) but its values (dedicated to environments) are stored in the diVerent place or repository that the helm chart itself

Helm

- Chart should be simple to read, containing yaml manifests for each required resource
- Secrets key-values should be generated randomly
- Deployment should have checksum injected to annotations

Kubernetes

- Configmap – Should contain sample json file content which will be mounted to the pod as config.json file to "app/" directory inside the pod
- Deployment template should have 3 replicas for development environment and 5 replicas for production environment
- Deployment should have RollingUpdate created that way to not create a Downtime
- ReplicaSet template should have sample Liveness and Readiness probes based on TCP port 8080
- ReplicaSet template should have environment variable POD_IP, which values will be gathered from pod's IP dynamically
- ReplicaSet template should let Pods start with arg "--spring.profiles.active=dev" for development environment and "--spring.profiles.active=prd" for production

- Every time pod is terminated there is command executed “wget http://localhost:8080/ /service/shutdown"
- ReplicaSet should have some resources allocation (requests and limits) for created pod
- Pod will expose ports 8080, 8081 and 8082
- Service template should have three ports exposed from pod and publish them to the equal values (port = targetPort) and service should be created as ClusterIP type
- Ingress template manifest should contain one host (one for development and one for production) of your choice
- Ingress template manifest should have three paths, depends on the port the service is exposing
  8080 - /api
  8081 - /logs
  8082 - /soap
- Secret template will have randomly generated key-values of your choice. These keys should be injected to the Deployment resource

Simplified flow